Final Reflection: Group 12

Parallelized Battery SOC Algorithm Evaluation Multitool

Project Repo: https://github.com/Dason-IsopodOverseer/Battery-Algorithm-Standardized-Testing-Tool

Members: Dason Wang, Ellen Xiong, Aidan McLean, Paarth Kadakia

Project under direct supervision of ***Phillip Kollmeyer, PhD***

Complexity Statement

Our project goals concern orchestration and software translation. We outline two main challenges:

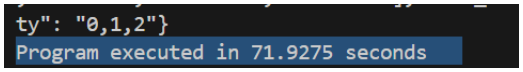
1) Evaluator Conversion and Operationalization (ECO) from MATLAB to Python was an utmost priority for us because we needed to appeal to two demographics: engineers and chemists familiar with MATLAB and data/ML scientists familiar with Python. *The main challenge here was* ***correctly interpreting existing MATLAB code*** *(given to us by our supervisor) and* ***translating it to Python,*** *then integrating it with our new distributed backend.* This translation also included ***refactoring of additional components*** written natively in MATLAB needing redesigns to fit our new API, such as the email system, to send results to users; the validation system, to ascertain the computability of submitted models; and the file I/O (previously continuously scanned for new files, now reworked to be explicitly called) and the passing of result data (from updating a global CSV file to passing JSON to backend). In short, much of the original architecture needed to be ***refactored,*** *and as we believe we have done,* ***improved.*** As shown in our interim video, MATLAB is slow, carries overhead, and difficult to do distributed computing with, and Python is the remedy.

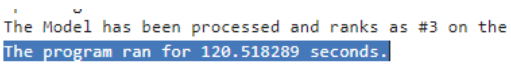
2) Parallelization in a cloud-orchestrated environment was a necessary pairing for the prior, simply because the larger MATLAB models took several days, which is a processing bottleneck which translation to Python could not alleviate. ***We designed a cloud-orchestrated processing pipeline*** that is hosted by Canada Compute (the host server, or parent scheduler) which controls workers from the combined high-power compute resources of Canada Compute on their Arbutus Cloud (our grant is not yet approved, so we haven't added them yet) and Google Cloud Compute. Due to the difficulty of this task (and other struggles outlined below), we incorporated a third-party service (https://coiled.io/) to handle containerization and orchestration logic (VM scheduling logic) through Dask data-frames. Additionally, we devised two sub-components of which were technically interesting: ***we have an allocation scheduler*** which doesn't handle queueing, but instead determines the ***ideal lifetime of allocated resources***; we also devised a ***complexity grader*** for submitted Python models in order to ***dynamically determine how much processing power to allocate to any submission*** *without needing to evaluate the submission itself, by a* ***heuristics*** *approach*. Sadly, the parallelization is not as robust as we had hoped (we failed to implement TLP, but did DLP instead) as some of what we had planned turned out to be technically infeasible, you can read about all our compromises in the detailed testing Jupyter Notebook in "Parallelization Workbench" of our Git Repo, **available here.**

Achievements

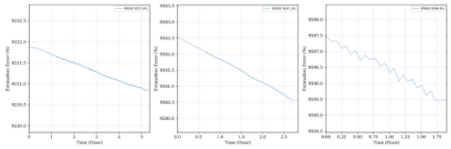
For our parallelization efforts, we found that it was **very effective** at reducing CPU load (thereby, server latency) at the cost of memory. We determined that memory was the bottlenecking component to parallelization effectiveness, especially to orchestrate very large models with embedded ***data.mat*** files. For our metrics, we found from testing that we were able to trade between incurring an extra **1.213 times additional avg. memory utilization burden,** in return for an awesome **57.692 times average CPU utilization reduction.** This reduces the total time the CPU spends on each model from upwards of 626 seconds to under 4 seconds (in our worst model during testing). *Parallelization does have costs*, which average to roughly **0.073 CAD** per model, and prolongs the total execution wait time of a model by **1.647 times** the normal amount. This means we make our users wait 1.65 times longer for their result to be returned (this is DLP, not TLP) and incur a $7 dollar cost per 100 users submitting a P0-P2 model (not considering neural nets with full GPU allocation). **In return, we expect to serve 57.7 times more users simultaneously!** Overall, we are very **pleased with these results!**

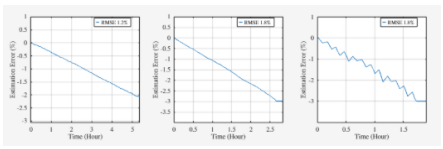
Converting and optimizing the evaluator (ECO) from MATLAB to Python let us achieve a significant speed boost in model evaluation. We chose to have the evaluator (ECO) focus solely on model evaluation computation and optimized how it was called in our system architecture by having it called by CLI through the backend when needed, which resulted in less overhead. Our efforts in optimization let us achieve an ECO evaluator that on average ran almost half as fast compared to the previous MATLAB evaluator (when directly compared with no parallelization). An example of this can be seen in this comparison test, where an SOC estimator model is fed into both the Python and MATLAB evaluators (equivalent models in their respective file formats). The following images show the program execution times (left Python evaluator, right MATLAB evaluator).





From this test, we found that the Python evaluator executed almost half as fast as the MATLAB evaluator. During validation of the converted Python evaluator against the original MATLAB implementation, we encountered several limitations. Despite careful alignment, the Python evaluator was unable to reproduce the MATLAB metrics within the target error tolerance of 1e-4. We suspect that this was the result of differences in floating-point values of the test data, random seed differences between MATLAB and Python as the main reason for our findings. Further details about this can be found in the "SRS and Project Plan" document. However, we achieved other behaviors that remained consistent across the MATLAB and Python evaluators. Though the metric values were different (error percentages), we found that the graphs produced by both evaluators exhibited similar shapes and trends. This indicates that while exact numerical equivalence was not achieved, the Python evaluator retained comparable performance and evaluative trends. The following is an example of a Python evaluator graph result (left) compared to a MATLAB evaluator graph result (right) of the same model.





We further verified that the Python evaluator produces model rankings consistent with those generated by the MATLAB evaluator. Models on the leaderboard are ordered using the Weighted Error metric, and when multiple models were evaluated, the Python implementation ranked them in the exact same order as the MATLAB version. This confirms that the conversion preserved the integrity of the ranking logic despite the numerical differences in the metric values. Further details and example of this can be found in the “Design and V&V” document.

P0 and P1 Completion

*The loss of a team member (Mohamed) was a huge setback for us, and consequently, any compromises to the completion tied to his leaving is marked with a **(MO) label** in the comments.*
*We have a detailed explanation for each underperforming component in our **updated SRS and Project Plan.** **You** can find it in the **Docs** directory of our Git Repo!*
Below is a list of our P0 & P1 requirements with percent completion (P0+ means extra priority):

Requirement	Priority	% FIN	Comments
Model Interpretation *What we now call ECO*	**P0+**	70%	(**MO**) Model Interpretation completed, but ran into various issues affirming the correctness, as discussed in V&V document. Several of our tests failed to be realized due to unforeseen difficulties. Loss of a team member assigned to maintain and understand MATLAB code resulted in the burden of interpretation being placed on one person, which slowed progress and hindered verification tests.

Parallelization	**P0+**	90%	Parallelization objectives mostly met; we implemented data-level parallelization (DLP) but did not accomplish task-level parallelization (TLP). See explanation in the notebook **here.**
Hosting a User Platform, with Submissions and Leaderboards	P0	100%	The main objective of the platform is to be able to submit your own model to make this website a central hub for comparing and contrasting SOC models. We were able to externally host the project using a free service in Canada Compute that allow for researchers to borrow their servers for hosting projects. This server pulls from a database of user submitted results, queries the ones that are made public, and ranks them upon a leaderboard. **All met.**
Model Submission File Reading System	P0	60%	(**MO**) **The core "failure" here was being unable to get a hosted version of our app which had MATLAB securely installed.** This requires a network license we were unable to obtain, though the basic working code is available in our **Legacy** folder on our Git repo; this code is **not** integrated with our backend because this task was assigned to **Mohammed,** and of course because we don't have a license. *See our updated SRS for the full breakdown.*
Building a Website Downloadable as a Web App, Local Processing	P0	0%	(**MO**) This was deprecated by Mohammed's leaving and further discussion with our supervisor. *See our updated SRS for the full breakdown.*
Security	P1	95 %	In our design document, we included 'Anomaly Response', which was not fully realized. While the system has extensive logging, including failed authentications, there is no explicit mechanism for anomaly response and notification. Additionally, we went beyond the basic requirements are now hosted on Canada Compute's Research servers with **custom hardened security rules**, including key-only authentication (no passwords), and allow access to only https or ssh requests from McMaster IP (public release pending).
Bug-Fixing	P1	100%	All foreseen issues and challenges were addressed, with reasonable compromises. All the bugs encountered in the legacy code, and our starting interpretations, were fixed.
Leaderboard Visualization	P1	50%	We overestimated the capabilities of our central server, as it turns out, it is very intensive to pass back data constantly from the orchestrated worker VMs to the dashboard to visualize evaluation results in real time on a time-series, therefore, "time-series with granularity up to second-by-second display" was not accomplished. We also did not have enough time to implement "statistical tool packages, graphing options." **As a compromise, we produced a graphical comparison of final evaluation results (metrics) so that researchers could more easily filter a model stronger (more accurate) in a particular category.** *This is far inferior to a time series but at least serves a visualization purpose.* Also, we did manage to get the very simple Search/Sort/Filter Functionality implemented, which we deem at least sufficient for 50% completion.

Highlights & Lowlights

Strangely, one of the happiest moments for us (which turned out to be completely irrelevant) was getting a ***full copy of MATLAB installed on our Ubuntu server which could be mounted into Docker***. Officially, it is recommended to install MATLAB from a pre-configured image with a network license, but since we didn't have that, we tried to install from a command-line (later, from a GUI). After much trial and error, we were able to complete installation and mount MATLAB to our running Docker container via *dependency injection* with MATLAB Engine in Python. This consumed a lot of development time, and yet we were unable to leverage this in our final product due to security/legal vulnerabilities (using a student license, on a server with a desktop environment) and the lack of legacy upkeep due to Mohammed's departure. Overall, this was a very **important learning experience** because it significantly deepened our expertise in IT infrastructure and various Linux distributions, and the whole team felt a **great sense of unity in attempting it and ultimately succeeding.** From the scarcity of helpful online discussions about this issue and the failure of AI assistants to give us anything useful, we believe that **we *might* be trailblazing** with the deployment of docker containers built from the ground up which injects MATLAB Engine in Python, we don't think anybody has done this before as we have done it!

Another great moment was seeing our hard work materialize when we first deployed our project and made it live at www.batterysocbenchmark.ca, and then showing it to our supervisor. This was **deeply gratifying**.

As for lowlights, ***Mohamed's November departure*** was our **biggest setback**, forcing us into overtime and role reshuffling that nearly jeopardized our P0s, primarily model submission. Each member of the team recalls feeling completely hopeless and overwhelmed. While we're proud of our perseverance, if we were to do this over, we would sincerely want a fifth team member to distribute our responsibilities and avoid similar burnout. For a long while, primarily during January, we also struggled immensely with the intricacies of our **Scheduler Component**, where we ran into the paradox of needing to grasp a model's expected complexity before we had evaluated it. We did eventually devise a heuristics-based grader, but this did not live up to our expectations of a highly sophisticated scheduler that might dynamically re-schedule during evaluation, for instance, remapping heavier workloads to more powerful CPUs on-the-fly. We also lagged behind on completing verification tests for our Python code. We spent so long deliberating on the perfect complicated solution, that nothing worked out, and we were forced to implement the simplest one instead.

Conclusion: What we learned

Our first highlight, as previously described, was a **very important lesson.** Why were we trying so hard to install MATLAB on our server, when it was questionable whether we could obtain a network license, and knowing beforehand the security risks and lack of backend integration? This made us realize that solving **new, interesting, and technically challenging** issues does not automatically amount to impressive engineering; it is always wise to make sure that our efforts are constantly aligned with the delivery of our priority tasks, especially when we have a manpower deficit.

We also learned a lot in designing and improving system architectures from the process of interpreting an inherited existing codebase. It was necessary for us to carefully evaluate the existing codebase to determine how it could be improved. We also had to identify which changes were needed and appropriate to align the system with our new goals, balancing functionality, performance, and maintainability. We learned the importance of system design when determining which components of the legacy codebase should be preserved and how to effectively integrate them into a redesigned/updated system architecture without introducing major complexity differences or breaking existing behavior.

We also learned much about deploying and hosting an application ourselves. We had to interface with unfamiliar (to some of us) technologies, such as nginx and pm2, and do much infrastructural setup, such as setting up SSH access for the team and proper installation of dependencies. A hurdle for us was the

acquisition and auto-renewal an SSL certificates for our application. Due to our strict firewall rules, the process was less straightforward than usual, and we had to search for the right third-party service that allowed us to maintain our current DNS provider (that doesn't have an API) and our firewall rules. Overall, these challenges gave us a deeper understanding of server administration, and the hosting, deployment, and maintenance of applications.